

TypeScript Cheat Sheet

The facts behind every question asked while reading the Obstacles doc, one line each

October 2026

Structures

1. `{}` is an object: the dict. `[]` is an array: the list. `new Map()` is a dict for any key type. `new Set()` is a set.
2. A plain object's keys are always strings; `o[1]` and `o["1"]` are the same key. `Map` keeps real key types.
3. `o.x` and `o["x"]` read a key; a missing key gives `undefined`, never an error.
4. `arr.length` for arrays and strings; `map.size` and `set.size` for collections.
5. `b = a` on an object or array makes two labels on ONE thing (same as Python). Copy with `[...a]`, `{...o}`, `structuredClone`.
6. A method chain reads left to right: `new Map().set(1, "a").get("1")` makes a Map, sets key `1`, then looks up the string `"1"` → `undefined`, because `Map` keeps key types (like a Python dict).
7. IDs from a URL or query are strings (`req.params.id` is `"42"`); IDs stored in a `Map` may be numbers. `map.get(req.params.id)` is then always `undefined`; convert first: `map.get(Number(req.params.id))`.

undefined, null, falsy

8. `undefined` = never set (missing key, missing argument, no `return`). `null` = deliberately empty.
9. "Falsy" is not a value; it is how a value behaves in `if`, `!`, `&&`, `||`.
10. Exactly eight falsy values: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy.
11. `undefined` is both: its value is `undefined` AND it behaves falsy. Both are true at once.
12. `[]`, `{}`, `new Set()` are TRUTHY (Python: falsy). `if (!items)` never fires for an empty array; use `items.length === 0`.
13. `"0"` and `"false"` are truthy (non-empty strings). Query-string values are strings.
14. `if (!x)` cannot tell "missing" from `"0"`; the precise missing test is `x === undefined` or `x == null`.

Equality

15. `===` compares type and value, no conversion. Python's `==` is the equivalent.
16. `==` converts both sides first ("coercion"), then compares: `0 == ""`, `5 == "5"`, `[] == false` are all `true`.
17. Always expect `===/!===`; flag any `==`. One exception: `x == null` deliberately catches both `null` and `undefined`.
18. A single `=` is assignment, never comparison. `if (o.status = "paid")` assigns and is always truthy.

NaN

19. `NaN` stands for Not a Number; its type is `number`.
20. It comes from failed numeric conversion or math: `Number("12px")`, `Number(undefined)`, `parseInt("abc")`, `0 / 0`, `"abc" * 2`.
21. Python raises on these; TS silently returns `NaN` and keeps going.
22. `NaN` is the only value not equal to itself: `NaN === NaN` is `false`. `if (x === NaN)` can never be true.
23. Every comparison with `NaN` is `false`: `NaN > 5`, `NaN < 5`, `NaN >= 0` are all `false`.
24. `NaN` is contagious: any arithmetic with it gives `NaN`.
25. A range check `if (size > 100)` lets `NaN` through because the comparison is `false`. Check positively: `if (!Number.isInteger(size) || size < 1 || size > 100)`.
26. Test with `Number.isNaN(x)`. The old global `isNaN(x)` coerces first and is wrong on strings.
27. `NaN` is falsy, so `if (!x)` catches it, but also catches `0`; use `Number.isInteger` when `0` is valid.

?? and ||

28. `a || b`: use `b` when `a` is any falsy value (like Python `a or b`).
29. `a ?? b`: use `b` only when `a` is `null` or `undefined`. Python has no equivalent.
30. `count || 10` turns a real `0` into `10`; `count ?? 10` keeps the `0`. If the left side can legitimately be `0`, `""` or `false`, `||` is wrong.
31. `x ??= 5` assigns only if nullish; `x ||= 5` if falsy.
32. In a condition (`if (a || b)`) `||` is ordinary boolean OR; the trap is only when `||` picks a value.

const, mutation, reassignment

33. `const` locks the label (the arrow from name to thing), not the contents of the thing.

34. `a.push(1)` is mutation: same array, contents change. Legal under `const`.
35. `a = [2]` is reassignment: label points at a new array. `TypeError` under `const`.
36. Python has no `const`; the two operations are in-place (`a.append`) vs rebinding (`a = [...]`).
37. `const orders; orders.sort(...)` still reorders the caller's array. `const` does not protect data.
38. Freezing contents is separate: `Object.freeze(a)` (runtime, shallow) or `readonly T[]` (type only).

Strings

39. Strings are immutable in TS and Python: no string method changes `s`; every one returns a new string.
40. `s.toUpperCase()`; on its own line does nothing. `s = s.toUpperCase()`; stores the result. Call $\rightarrow$ assign, always.
41. `s = s.trim()` is a reassignment, so `s` needs `let`; under `const` use a new name (`const clean = s.trim()`).
42. `s[0] = "x"` is silently ignored (Python: `TypeError`). Build a new string: `"x" + s.slice(1)`.

Arrays: new vs in place

43. Return a NEW array (must be assigned or the result is dropped): `map`, `filter`, `slice`, `concat`, `flat`, `flatMap`, `toSorted`, `toReversed`.
44. Return a value, not an array: `reduce`, `find`, `findIndex`, `some`, `every`, `includes`, `indexOf`, `join`.
45. Change the array IN PLACE: `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, `fill`.
46. `arr.filter(x => x > 0)`; alone does nothing (like `sorted(xs)` alone in Python). `const pos = arr.filter(...)` catches it.
47. `const sorted = arr.sort()` is NOT a copy: `.sort()` mutates and returns the same array. Use `[...arr].sort()`.
48. `arr[-1]` is `undefined`; last element is `arr.at(-1)` or `arr[arr.length - 1]`.
49. `arr[i]` past the end is `undefined`, never an exception.

The arrow `=>`

50. `x => x > 0` is `lambda x: x > 0`: parameter on the left, returned expression on the right.
51. `=>` reads "returns". There is no `if` in the arrow; it returns the boolean `x > 0`.
52. The deciding happens in the method: `filter` wraps your condition in its own loop and `if`, and keeps `x` when your function returned `true`.
53. `arr.filter(x => x > 0)` reads: go through `arr`, call `x => x > 0` on each element, keep the ones where it returns `true`.
54. Two parameters need parentheses: `(a, b) => a - b`. No parameters: `() => 42`.
55. Braces make a block body and need an explicit `return`: `x => { x * 2 }` returns `undefined`; `x => x * 2` returns `x * 2`.
56. An arrow returning an object literal needs parentheses: `() => ({ a: 1 })`.
57. What the method does with the arrow's return value depends on the method: `filter` keeps if `true`; `map` stores the value; `find` returns the first `x` where `true`; `sort` uses the sign of the number; `some` is `true` if any was `true`.
58. If/else inside an arrow is the ternary `x => x > 0 ? "pos" : "neg"` (Python `a if c else b`), or a block with real `if` and `return`.

Sorting

59. `.sort()` with no argument converts elements to strings and sorts alphabetically: `[10, 9, 1].sort()` is `[1, 10, 9]`.
60. A comparator is a function of two elements returning a number: negative = `a` first, positive = `b` first, zero = keep order.
61. `(a, b) => a - b` is ascending numeric; `(a, b) => b - a` is descending; `(a, b) => a.localeCompare(b)` for strings.
62. Python's `key=lambda x: x.score` says what to sort by; TS says how to compare: `(a, b) => a.score - b.score`.
63. A comparator returning a boolean (`a.score > b.score`) is a bug: unreliable order. Must return a number.

Loops

64. `for (const x of arr)` gives VALUES. `for (const k in obj)` gives KEYS, always as strings. Mnemonic: `of` = values of, `in` = keys in.
65. `for k, v in d.items()` $\rightarrow$ `for (const [k, v] of Object.entries(d))`. `Object.entries` turns `{a: 1}` into `[["a", 1]]`; `const [k, v]` unpacks each pair.
66. `for k in d` $\rightarrow$ `for (const k of Object.keys(d))`. `for v in d.values()` $\rightarrow$ `for (const v of Object.values(d))`.
67. `for...in` on an ARRAY gives indices `"0"`, `"1"` as strings, not elements. Bug; flag it.
68. A `Map` iterates directly: `for (const [k, v] of map)`, no `Object.entries`.
69. `forEach` cannot `break`, ignores `return`, and does not wait for `await`. `for...of` does all three.